

Introduction to SystemVerilog HDL design

Today's Agenda

Intended topics for today's session

- Learning by example – A first taste: A series of basic sequential SystemVerilog designs
- Conclusion and coding guidelines

FPGA-based SoC Design

International Master of Science in Electrical Engineering

Prof. Dr. C. Jakob

University of Applied Sciences Darmstadt

h_da

Faculty of Electrical Engineering and Information Technology

fbeit

Introduction to SystemVerilog HDL design

Objectives

By the end of this lecture you will be able to ...

- design simple clock-synchronous logic in SystemVerilog

FPGA-based SoC Design

International Master of Science in Electrical Engineering

Prof. Dr. C. Jakob

University of Applied Sciences Darmstadt

h_da

Faculty of Electrical Engineering and Information Technology

fbeit

Introduction to SystemVerilog HDL design

Recommended Readings

Textbooks, Application Notes, White Papers ...

- Sutherland, S., “RTL Modeling with SystemVerilog for Simulation and Synthesis: Using SystemVerilog for ASIC and FPGA Design”, CreateSpace Independent Publishing Platform, 2017
- Spear, C., “SystemVerilog for Verification: A Guide to Learning the Testbench Language Features”, Springer, 3rd edition, 2012.

FPGA-based SoC Design

International Master of Science in Electrical Engineering

Prof. Dr. C. Jakob

University of Applied Sciences Darmstadt

h_da

Faculty of Electrical Engineering and Information Technology

fbeit

Introduction to SystemVerilog HDL – Part #3

International Master of Science in Electrical Engineering

Prof. Dr. C. Jakob

University of Applied Sciences Darmstadt

h_da

Faculty of Electrical Engineering and Information Technology

fbeit

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

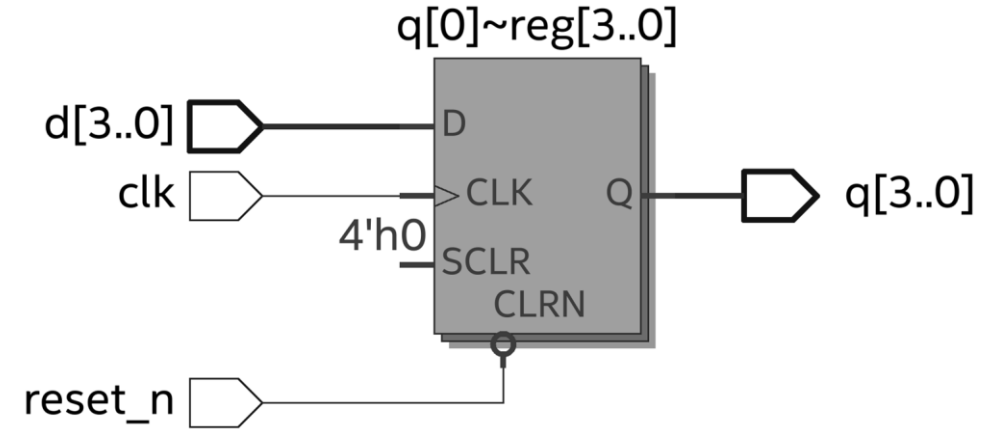
(PIP0) Registers in SystemVerilog

Introduction to SystemVerilog

Asynchronously resettable Register

```
1. module dff_4bit_load_aset(  
2.     input logic [3:0] d, clk, reset_n,  
3.     output logic [3:0] q  
4. );  
5.     always_ff@(posedge clk, negedge reset_n)  
6.         if(reset_n == 0)  
7.             q <= 0;  
8.         else  
9.             q <= d;  
10. endmodule
```

1/1

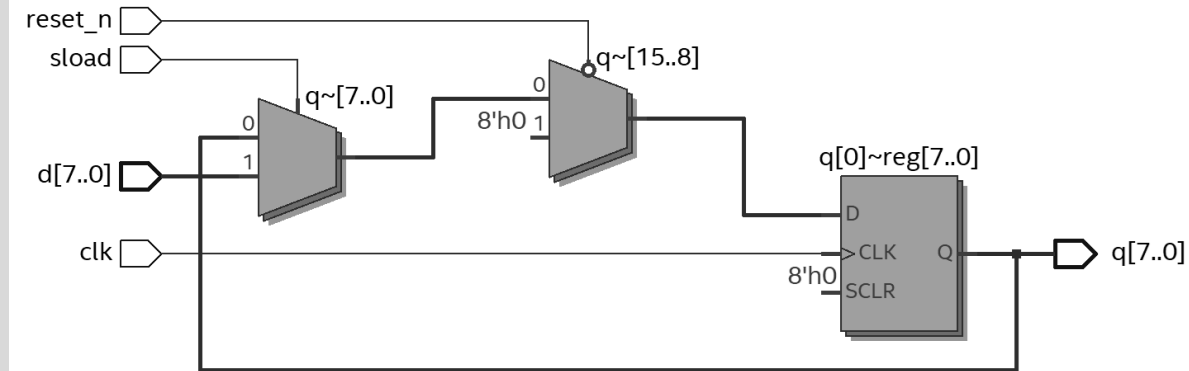


[SystemVerilog] Source Code: dff_4bit_load_aset.sv

Synchronously resettable Register with synchronous load

```
1. module reg_pipo_8bit_sreset_sload(
2.     input logic [7:0] d, input logic clk, reset_n,
3.     sload, output logic [7:0] q
4. );
5.
6.     always_ff@(posedge clk)
7.         if(reset_n == 0)
8.             q <= 0;
9.         else if(sload == 1)
10.            q <= d;
11.         else
12.            q <= q;
13.
14. endmodule
```

1/1



[SystemVerilog] Source Code: reg_pipo_8bit_sreset_sload.sv

Notes

- PIPO register stands for parallel-in parallel-out register, thus we've got 8 flip-flops which are loaded and read in parallel.

MCU Peripheral Configuration and Status Registers

```
1. logic [31:0] reg_0 = 0;
2. // timer0 peripheral configuration register placed
3. // address 0x40020014
3. always_ff@(posedge clk)
4.     if(reset_n == 0)
5.         reg_0 <= 0;
6.     else if(wren)
7.         reg_0 <= data;
8.
9.
10. assign TIMER_0_ENABLE      = reg_0[0];
11. assign TIMER_0_OVF_EN     = reg_0[1];
12. assign TIMER_0_OVF_IRQ_EN = reg_0[2];
```

1/1

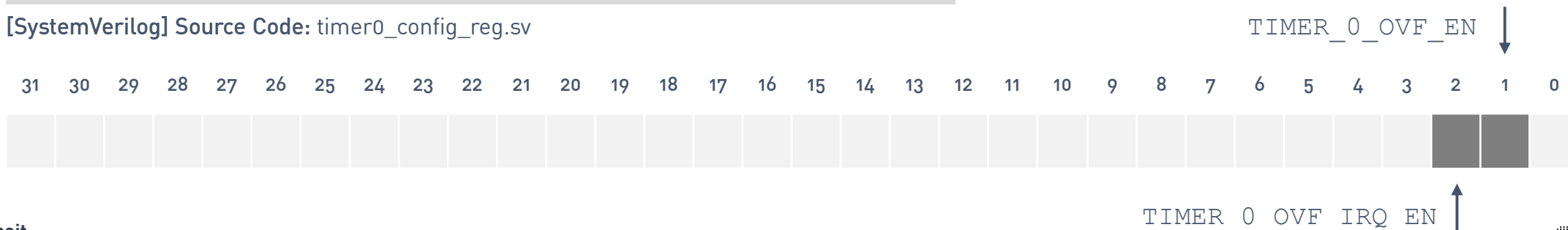
Notes

- We will get back to this structure when we are designing status and control registers for microcontroller peripherals ...

```
* ((unsigned int *) 0x40020014) |= 1 << 1;
* ((unsigned int *) 0x40020014) |= 1 << 2;
...
```

You are defining the register layout as well as the corresponding meaning/functionality of the respective register bits.

[SystemVerilog] Source Code: timer0_config_reg.sv



Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

(SISO) Registers in SystemVerilog

Introduction to SystemVerilog

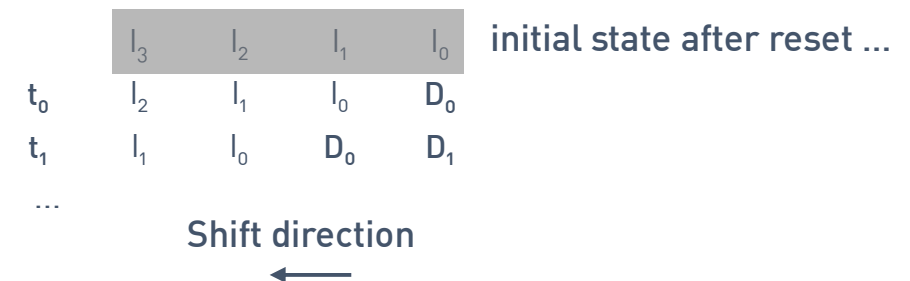
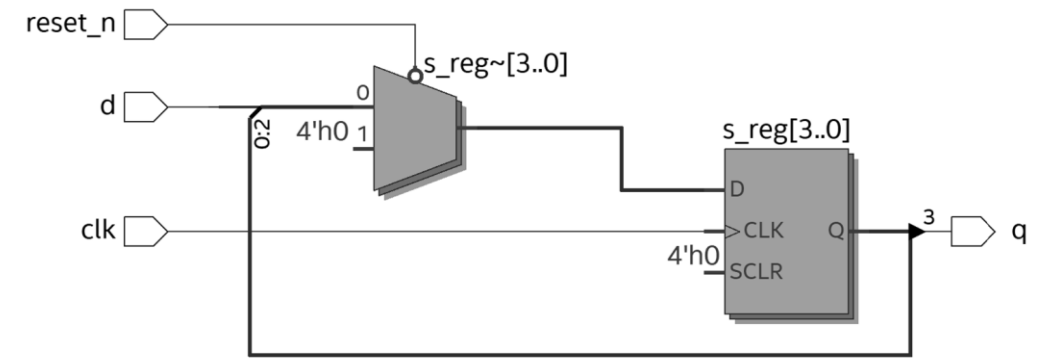
Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Synchronously resettable Shift-Register, serial in, serial out

```
1. module shift_reg_4bit_aset(
2.     input logic d, clk, reset_n,
3.     output logic q
4. );
5.     logic [3:0] s_reg = 0;
6.
7.     always_ff@(posedge clk)
8.         if(reset_n == 0)
9.             s_reg <= 0;
10.        else
11.            s_reg <= {s_reg[2:0], d}; left-shift operation
12.
13.        assign q = s_reg[3];
14.
15.    endmodule
```

1/1



[SystemVerilog] Source Code: shift_reg_4bit_aset.sv

Notes

- Left-Shift operation, { , } - Concatenation Operator.

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Synchronously resettable Shift-Register, serial in, serial out

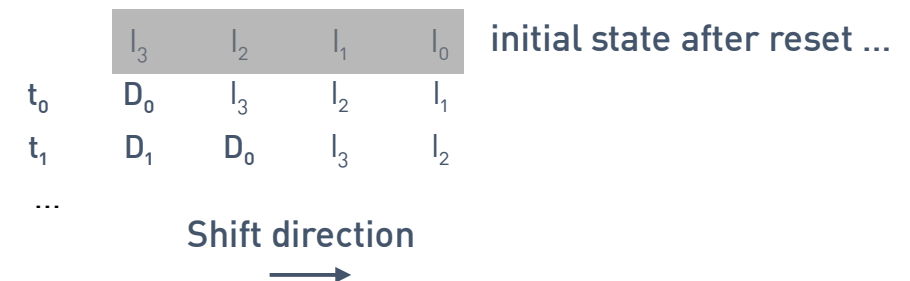
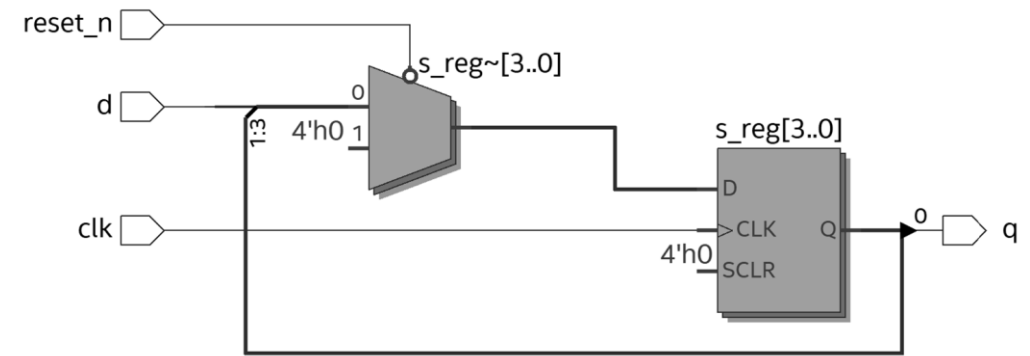
```
1. module shift_reg_4bit_aset(
2.     input logic d, clk, reset_n,
3.     output logic q
4. );
5.     logic [3:0] s_reg = 0;
6.
7.     always_ff@(posedge clk)
8.         if(reset_n == 0)
9.             s_reg <= 0;
10.        else
11.            s_reg <= {d,s_reg[3:1]}; right-shift operation
12.
13.        assign q = s_reg[0];
14.
15.    endmodule
```

new data is shifted in from the LHS:

[SystemVerilog] Source Code: shift_reg_4bit_aset.sv

Notes

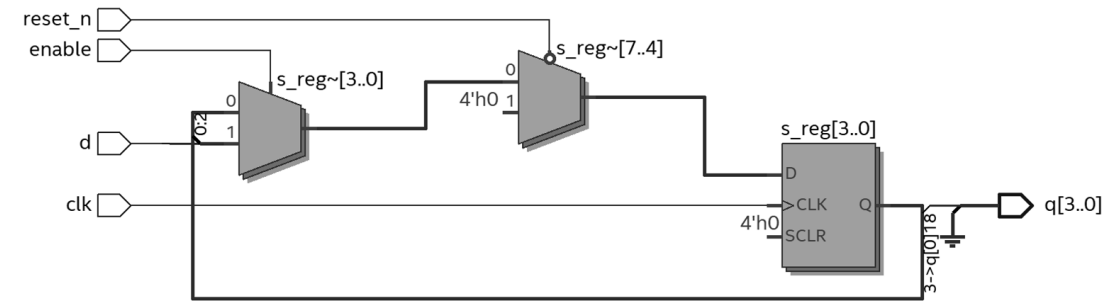
- Right-Shift operation, { , } - Concatenation Operator.



Synchronously resettable Shift-Register, serial in, serial out and shift-enable

```
1. module shift_reg_4bit_aset(
2.     input logic d, clk, reset_n, enable,
3.     output logic q
4. );
5.     logic [3:0] s_reg = 0;
6.
7.     always_ff@(posedge clk)
8.         if(reset_n == 0)
9.             s_reg <= 0;
10.        else if(enable) ← shift-enable
11.            s_reg <= {s_reg[2:0], d};
12.
13.    assign q = s_reg[3];
14.
15. endmodule
```

1/1



[SystemVerilog] Source Code: shift_reg_4bit_aset.sv

Notes

- Left-Shift operation, { , } - Concatenation Operator.

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

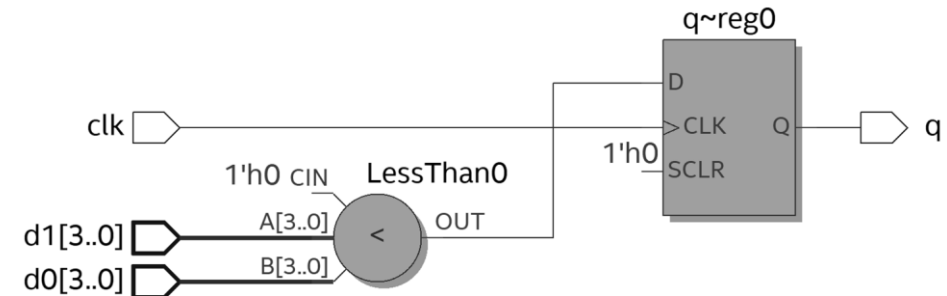
Combinational Logic with registered Outputs

Introduction to SystemVerilog

Comparator with reg. output - Conditional assignment

```
1. module comp_gt_4bit_reg(  
2.     input logic [3:0] d0, d1, input logic clk,  
3.     output logic q  
4. );  
5.  
6.     always_ff@(posedge clk)  
7.         q <= (d0 > d1) ? 1 : 0;  
8.  
9. endmodule
```

1/1



[SystemVerilog] Source Code: comp_gt_4bit_reg.sv

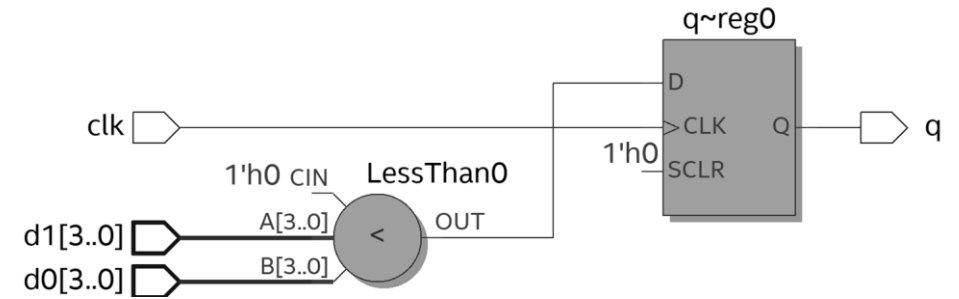
Notes

- The comparator stage is combined with another delay flip-flop ...
- So by placing combinational logic, arithmetic operations or comparison logic into an always_ff block, we create Delay Flip-Flops along with the combinational logic. In all cases the logic is driving the inputs of the DFF's ...

Comparator with reg. output - Conditional assignment

```
1. module comp_gt_4bit_reg(  
2.     input logic [3:0] d0, d1, input logic clk  
3.     output logic q  
4. );  
5.  
6.     always_ff@(posedge clk)  
7.         if (d0 > d1)  
8.             q <= 1;  
9.         else  
10.            q <= 0;  
11.  
12. endmodule
```

1/1



[SystemVerilog] Source Code: comp_gt_4bit_reg.sv

Notes

- This is just an alternative description of the previous design ...

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

When things go wrong – Using blocking statements in the wrong context ...

Introduction to SystemVerilog

When things go wrong – Using blocking statements in the wrong context ...

- The actual intention is to model two Delay-Flip-Flops connected in sequence ...

When things go wrong – Using blocking statements in the wrong context ...

```
1. module shift_reg_2bit_sreset(  
2.     input logic d, clk, reset_n,  
3.     output logic q  
4. );  
5.     logic [1:0] shift_reg = 0;  
6.  
7.     always_ff@(posedge clk)  
8.         if(reset_n == 0)  
9.             shift_reg = 0;  
10.        else begin  
11.            shift_reg[0] = d;  
12.            shift_reg[1] = shift_reg[0];  
13.        end  
14.  
15.    assign q = shift_reg[1];  
16.  
17. endmodule
```

1/1

Notes

- The actual intention is to model **two DFFs connected in sequence**.

[SystemVerilog] Source Code: shift_reg_2bit_sync_reset.sv

When things go wrong – Using blocking statements in the wrong context ...

```
1. module shift_reg_2bit_sreset(
2.     input logic d, clk, reset_n,
3.     output logic q
4. );
5.     logic [1:0] shift_reg = 0;
6.
7.     always_ff@(posedge clk) ← clock-sync., sequential
8.         if(reset_n == 0)                                procedural block
9.             shift_reg = 0;
10.    else begin ← blocking assignment
11.        shift_reg[0] = d;                                operator
12.        shift_reg[1] = shift_reg[0];
13.    end
14.
15.    assign q = shift_reg[1];
16.
17. endmodule
```

1/1

Notes

- The actual intention is to model **two DFFs connected in sequence**.

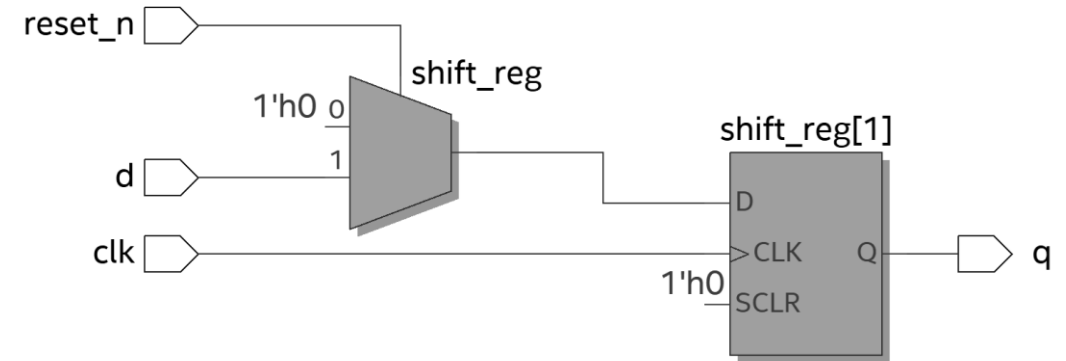
[SystemVerilog] Source Code: shift_reg_2bit_sync_reset.sv

When things go wrong – Using blocking statements in the wrong context ...

```
1. module shift_reg_2bit_sreset(
2.     input logic d, clk, reset_n,
3.     output logic q
4. );
5.     logic [1:0] shift_reg = 0;
6.
7.     always_ff@(posedge clk)
8.         if(reset_n == 0)
9.             shift_reg = 0;
10.        else begin
11.            shift_reg[0] = d;
12.            shift_reg[1] = shift_reg[0];
13.        end
14.
15.    assign q = shift_reg[1];
16.
17. endmodule
```

The **blocking assignment** operator causes the assignments to be performed as if in sequential order.

1/1



Notes

- The actual intention is to model **two DFFs connected in sequence**.
- Please note the synthesis result ...

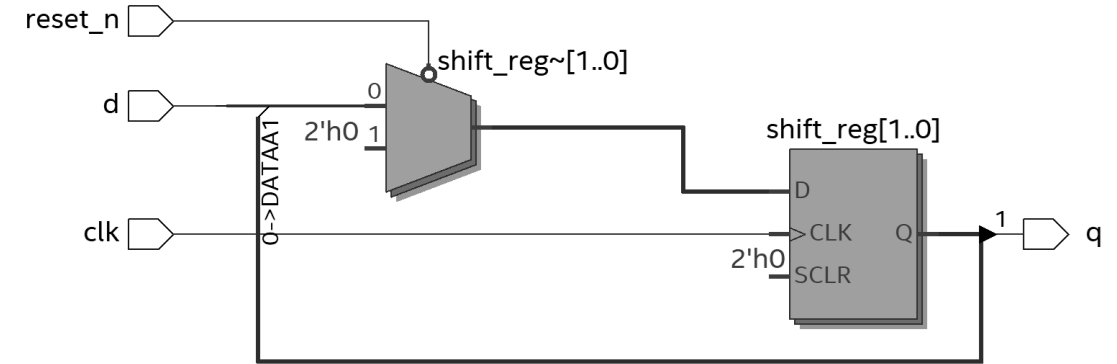
[SystemVerilog] Source Code: shift_reg_2bit_sync_reset.sv

When things go wrong – Using blocking statements in the wrong context ...

```
1. module shift_reg_2bit_sreset(
2.     input logic d, clk, reset_n,
3.     output logic q
4. );
5.     logic [1:0] shift_reg = 0;
6.
7.     always_ff@(posedge clk)
8.         if(reset_n == 0)
9.             shift_reg <= 0;
10.        else begin
11.            shift_reg[0] <= d;
12.            shift_reg[1] <= shift_reg[0];
13.        end
14.
15.    assign q = shift_reg[1];
16.
17. endmodule
```

1/1

← non-blocking assignment operator



Note

- The code is almost the same as before, but now with non-blocking assignment operators what finally leads to the desired synthesis result.

[SystemVerilog] Source Code: shift_reg_2bit_sync_reset.sv

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Counters in SystemVerilog

Introduction to SystemVerilog

Introduction to SystemVerilog

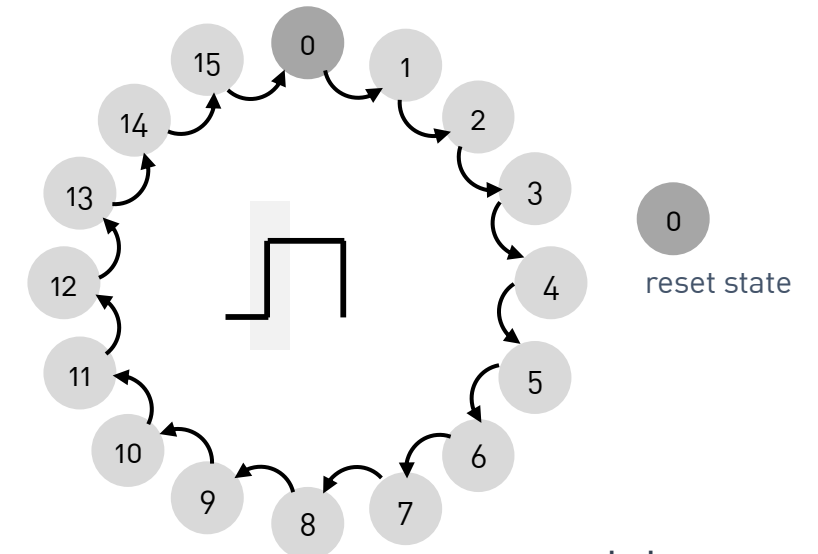
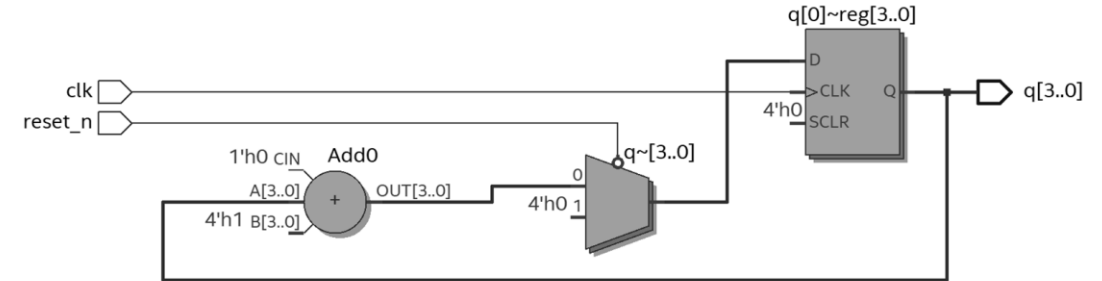
h_da – fbeit - FPGA-based SoC Design

Synchronous Up-Counter Version '1'

```
1. module sync_counter_v1(  
2.     input logic clk, input logic reset_n,  
3.     output logic [3:0] q  
4. );  
5.     always_ff@(posedge clk)  
6.         if(reset_n == 0)  
7.             q <= 0;  
8.         else  
9.             q <= q + 1;  
10. endmodule
```

1/1

[SystemVerilog] Source Code: sync_counter_v1.sv



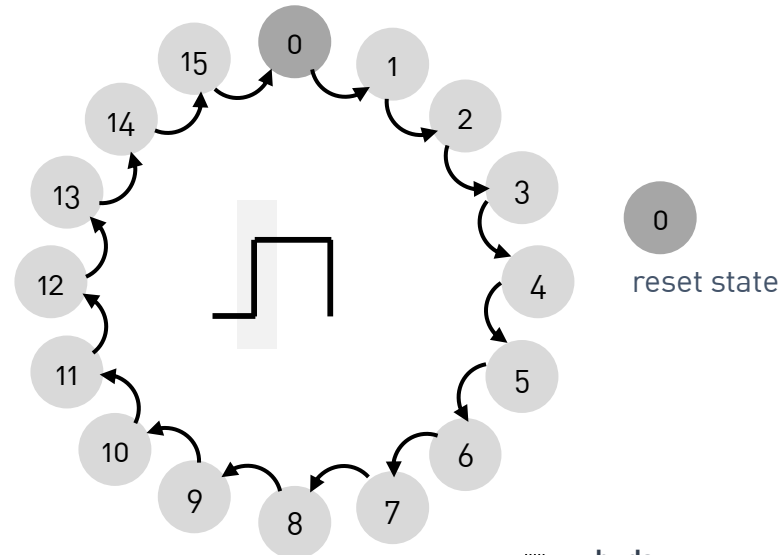
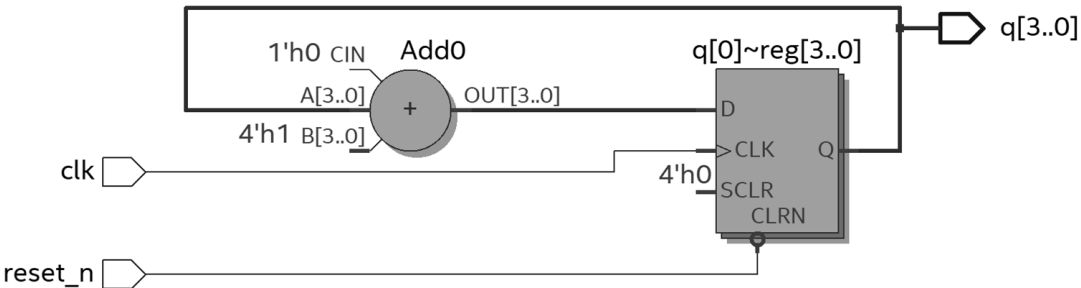
Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Synchronous Up-Counter Version '2'

```
1. module sync_counter_v2(
2.     input logic clk, input logic reset_n,
3.     output logic [3:0] q
4. );
5.     always_ff@(posedge clk, negedge reset_n)
6.         if(reset_n == 0)
7.             q <= 0;
8.         else
9.             q <= q + 1;
10. endmodule
```

[SystemVerilog] Source Code: sync_counter_v2.sv



Introduction to SystemVerilog

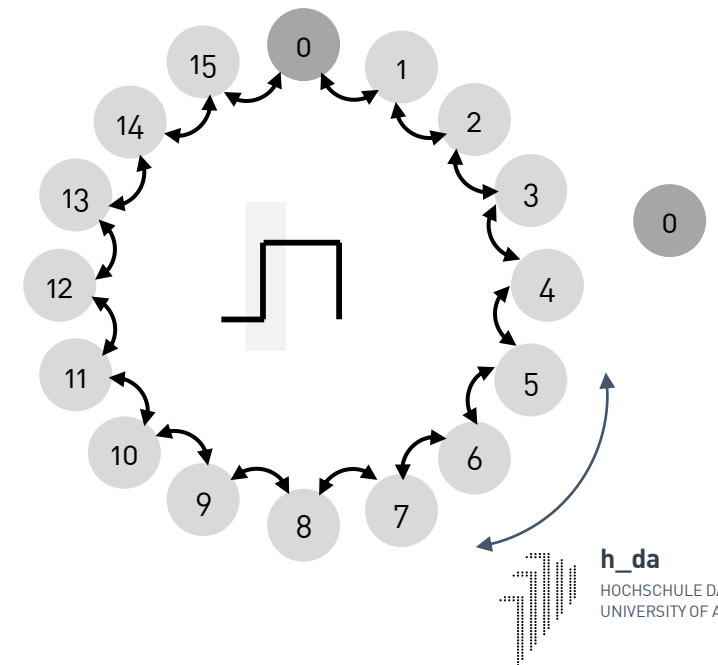
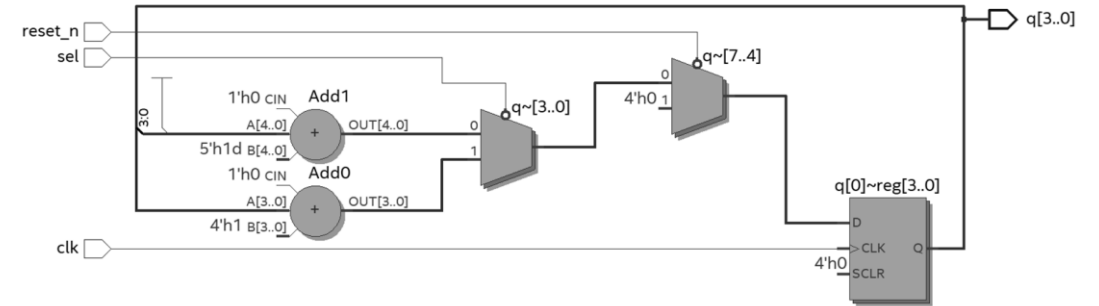
h_da – fbeit - FPGA-based SoC Design

Synchronous Up-Counter Version '3'

```
1. module sync_counter_v3(
2.     input logic clk, reset_n, sel,
3.     output logic [3:0] q
4. );
5. always_ff@(posedge clk)
6.     if(reset_n == 0)
7.         q <= 0;
8.     else if(sel == 0) ← select determines the
9.         q <= q + 1;      counting direction
10.    else
11.        q <= q - 1;
12.
13. endmodule
```

[SystemVerilog] Source Code: sync_counter_v3.sv

1/1



Introduction to SystemVerilog

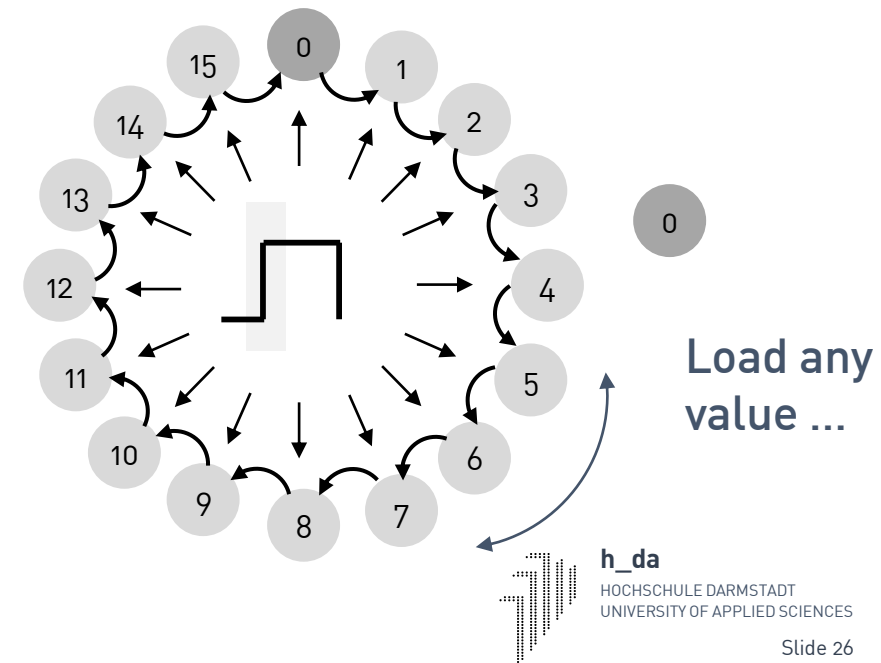
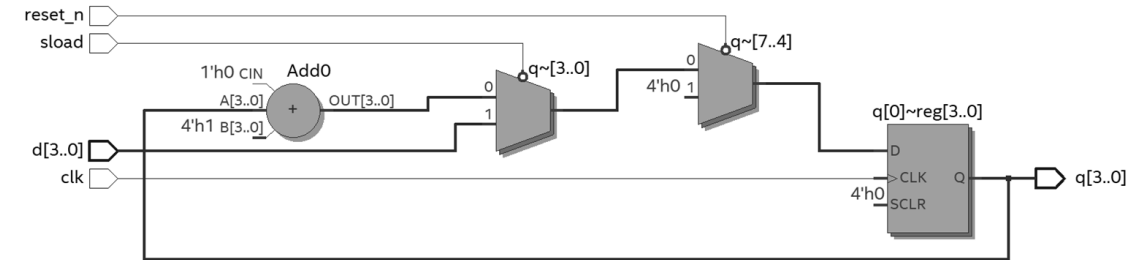
h_da – fbeit - FPGA-based SoC Design

Synchronous Up-Counter Version '4'

```
1. module sync_counter_v4(  
2.     input logic clk, reset_n, sload,  
3.     input logic [3:0] d, output logic [3:0] q  
4. );  
5.     always_ff@(posedge clk)  
6.         if(reset_n == 0)  
7.             q <= 0;  
8.         else if(sload == 0) ← load new start value ...  
9.             q <= d;  
10.        else  
11.            q <= q + 1;  
12.  
13. endmodule
```

1/1

[SystemVerilog] Source Code: sync_counter_v4.sv



Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Synchronous Up-Counter Version '5'

```
1. module sync_counter_v5(
2.     input logic clk, reset_n,
3.     output logic [3:0] q, output logic ovf
4. );
5.     always_ff@(posedge clk)
6.         if(reset_n == 0) begin
7.             q <= 0; ovf <= 0;
8.         end
9.         else begin
10.            q <= q + 1; ovf <= &q;
11.        end
12.    endmodule
```

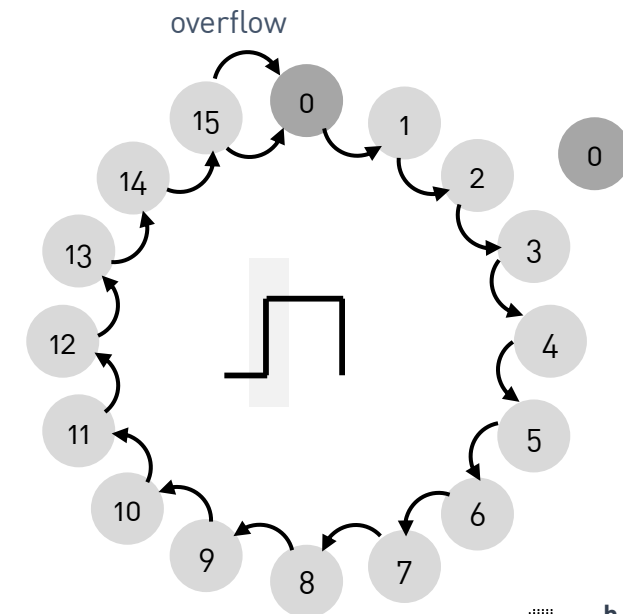
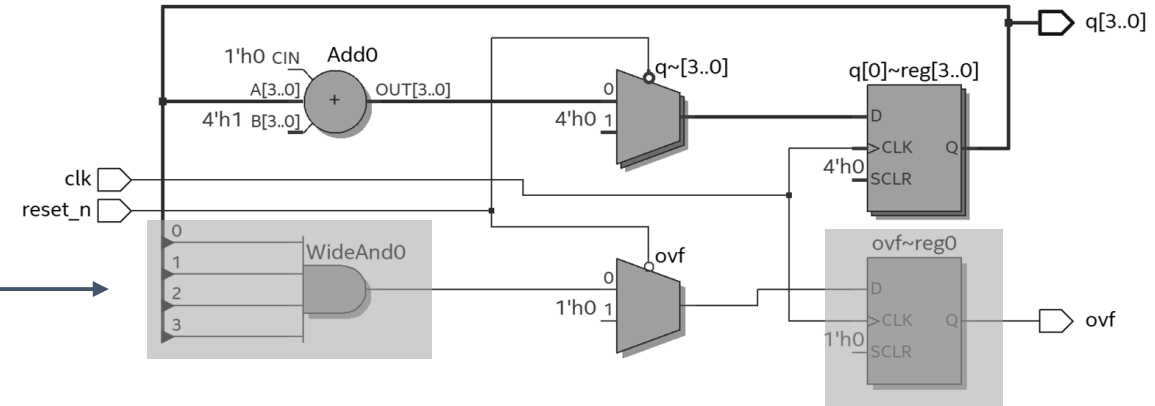
1/1

overflow status signal

[SystemVerilog] Source Code: sync_counter_v5.sv

Notes

- In the above example, the SystemVerilog reduction '&' operator is used to generate the overflow signal ...
- The overflow signal is assigned within the procedural always_ff block. This creates an additional output flip-flop that registers the actual overflow signal.



Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Synchronous Up-Counter Version '6'

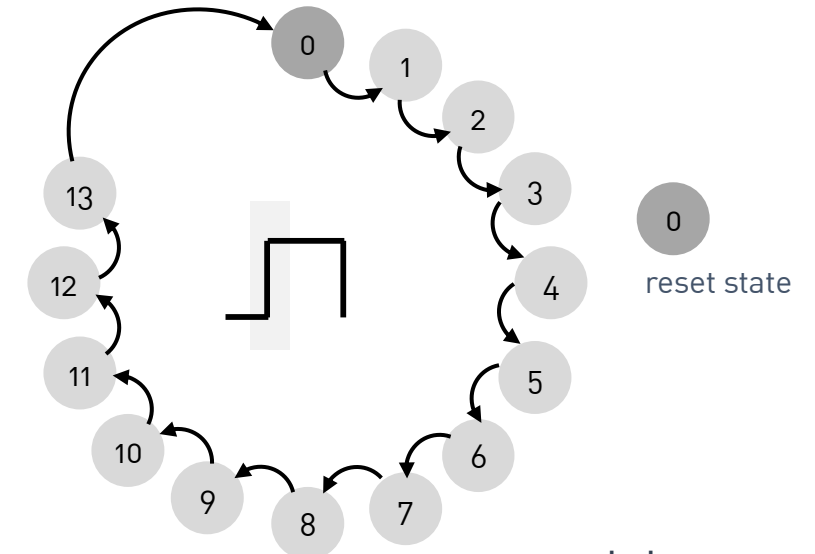
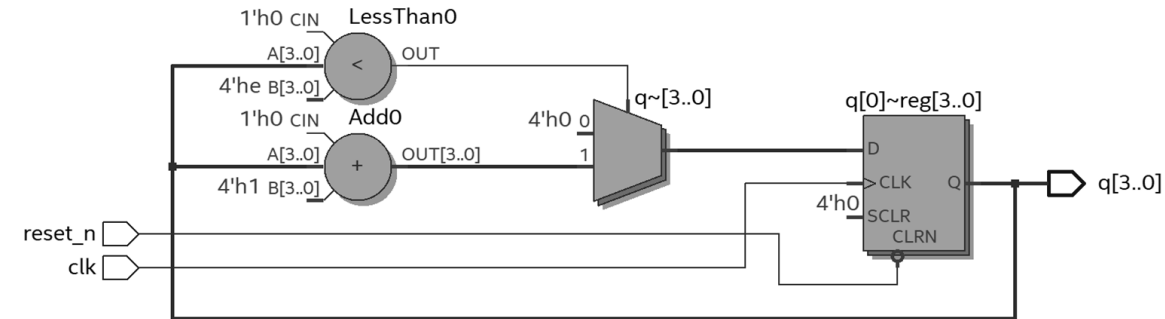
```
1. module sync_counter_v6(  
2.     input logic clk, input logic reset_n,  
3.     output logic [3:0] q  
4. );  
5.     always_ff@(posedge clk, negedge reset_n)  
6.         if(reset_n == 0)  
7.             q <= 0;  
8.         else if(q < 13) → 13 is part of the  
9.             q <= q + 1;           counting sequence  
10.        else  
11.            q <= 0;  
12. endmodule
```

1/1

[SystemVerilog] Source Code: sync_counter_v6.sv

Notes

- The resulting counting sequence is as follows: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 0, 1, 2, 3, 4, ...



Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Synchronous Up-Counter Version '7'

```
1. module sync_counter_v7(
2.     input logic clk, input logic reset_n,
3.     output logic [3:0] q
4. );
5.     always_ff@(posedge clk, negedge reset_n)
6.         if(reset_n == 0)
7.             q <= 0;
8.         else
9.             q <= (q + 1) % 14;
10. endmodule
```

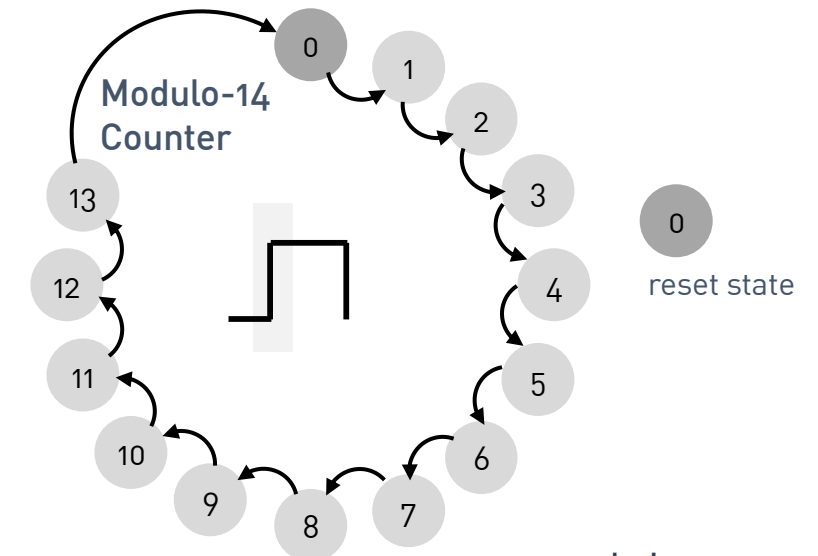
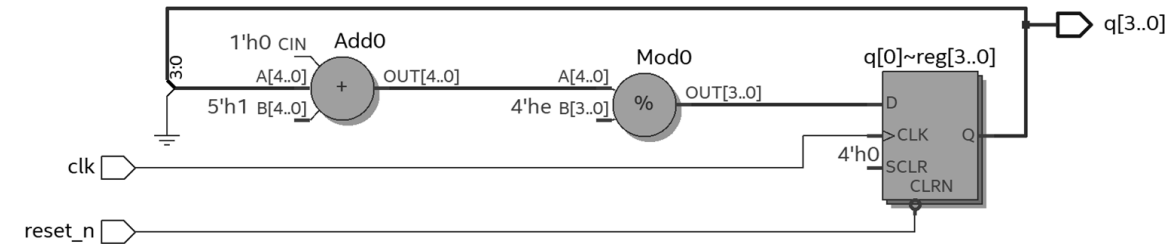
1/1

the counting sequence comprises of 14 numbers ...

[SystemVerilog] Source Code: sync_counter_v7.sv

Notes

- The resulting counting sequence is as follows: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 0, 1, 2, 3, 4, ...



Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Synchronous Up-Counter Version '8'

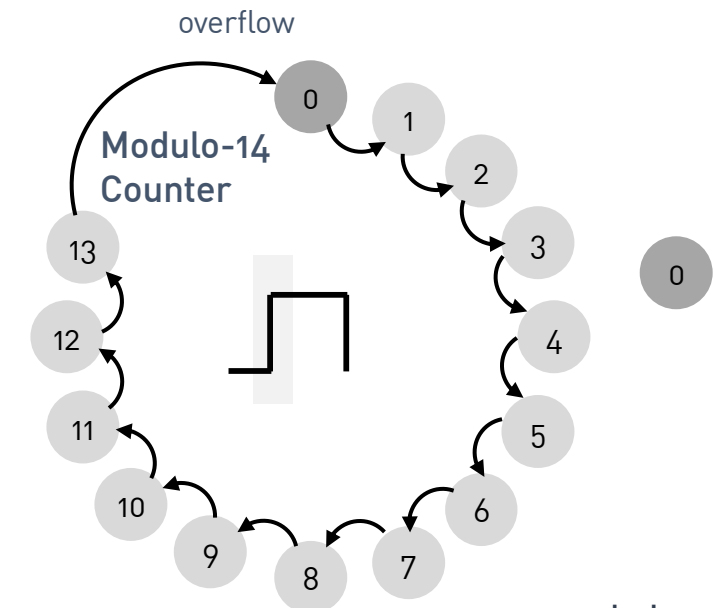
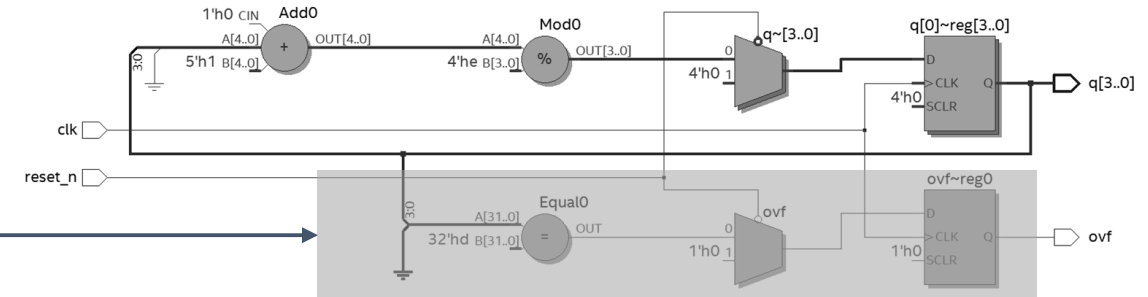
```
1. module sync_counter_v8(  
2.     input logic clk, reset_n,  
3.     output logic [3:0] q, output logic ovf  
4. );  
5.     always_ff@(posedge clk)  
6.         if(reset_n == 0) begin  
7.             q <= 0; ovf <= 0;  
8.         end  
9.         else begin  
10.            q <= (q + 1) % 14; ovf <= (q == 13) ? 1:0;  
11.        end  
12.  
13. endmodule
```

comparator with registered output

[SystemVerilog] Source Code: sync_counter_v8.sv

Notes

- Here, a **conditional assignment** is used to generate the overflow signal ...



h_da – fbeit - FPGA-based SoC Design

```
1. module gray_count_4bit(
2.     input logic clk, reset_n,
3.     output logic [3:0] q
4. );
5. logic [3:0] bin_count = 0;
6.
7. always_ff@(posedge clk)
8.     if(reset_n == 0)
9.         bin_count <= 0;
10.    else
11.        bin_count <= bin_count + 1;
12.
13. assign q = (bin_count >> 1) ^ bin_count;
14.
15. endmodule
```

Prof. Dr. C. Jakob



- Gray code after Frank Gray, is an ordering of the binary numeral system such that two successive values differ in only one bit:

 **h_da**
HOCHSCHULE DARMSTADT
UNIVERSITY OF APPLIED SCIENCES

Slide 31

SystemVerilog – A first set of coding guidelines

International Master of Science in Electrical Engineering

Prof. Dr. C. Jakob

University of Applied Sciences Darmstadt

h_da

Faculty of Electrical Engineering and Information Technology

fbeit

A first set of coding guidelines

- When modeling sequential (clock synchronous) logic, use non-blocking assignments!
- When modeling combinational logic with an `always_comb` block, use blocking assignments.
- When modeling both sequential and combinational logic within the same `always_ff` block, use non-blocking assignments.
- Do not mix blocking and non-blocking assignments within a single `always` block.
- Do not make assignments to the same variable from more than one `always` block.
- If using SystemVerilog for RTL design, we use the SystemVerilog logic data type to declare all point-to-point nets, for all variables (logic driven by `always` blocks), for all input ports as well as for all output ports.